

# Transaction-Conc epts

# What is a transaction?

- In the context of databases and data storage systems, a **transaction** is any operation that is treated as a single unit of work, which either completes fully or does not complete at all, and leaves the storage system in a consistent state.
- The classic example of a transaction is what occurs when you withdraw money from your bank account. Either the money has left your bank account, or it has not — there cannot be an in-between state.

# Transactions explained

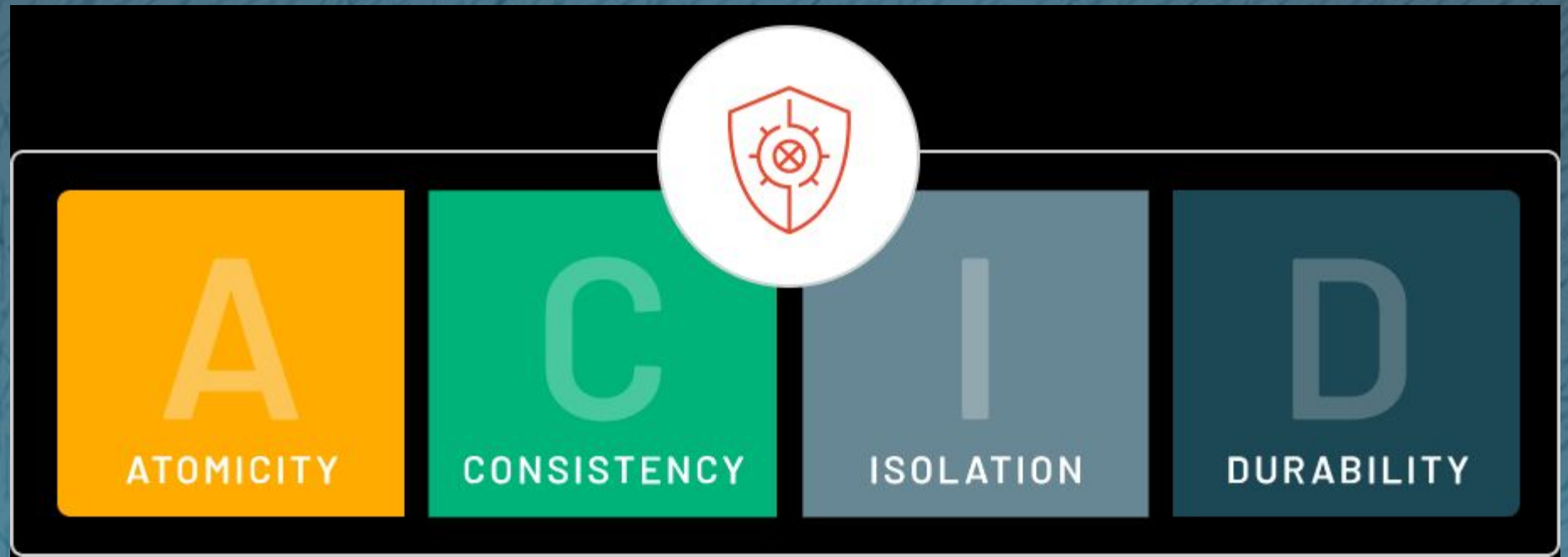
- At their most basic level, database transactions are a group of database read and write operations that have been completed successfully according to DBMS definitions (e.g., defined transaction criteria). There are two main types of transactions:
- **Single transactions:** A single transaction refers to a series of one or more database operations resulting in one action, completed successfully. Once complete, the transaction is accepted and can be found in a transaction log. A common example of a single transaction is the withdrawal of money from an ATM.
- **Multi-transactions:** A multi-transaction, sometimes called a distributed transaction, consists of multiple, interdependent transactions spread across different databases and systems (e.g., distributed systems). Records of these transactions are also found in a transaction log. An example of these transactions includes transferring money from one account to another.

# ACID transactions

- In these situations, a standard adherence to DBMS definitions is not enough, and ACID transactions are required.
- ACID is an acronym that stands for atomicity, consistency, isolation, and durability (ACID).
- Together, ACID properties ensure that a set of database operations (grouped together in a transaction) leave the database in a valid state even in the event of unexpected errors.
- Further, ACID transactions provide the level of transactional guarantees that many regulatory agencies require.

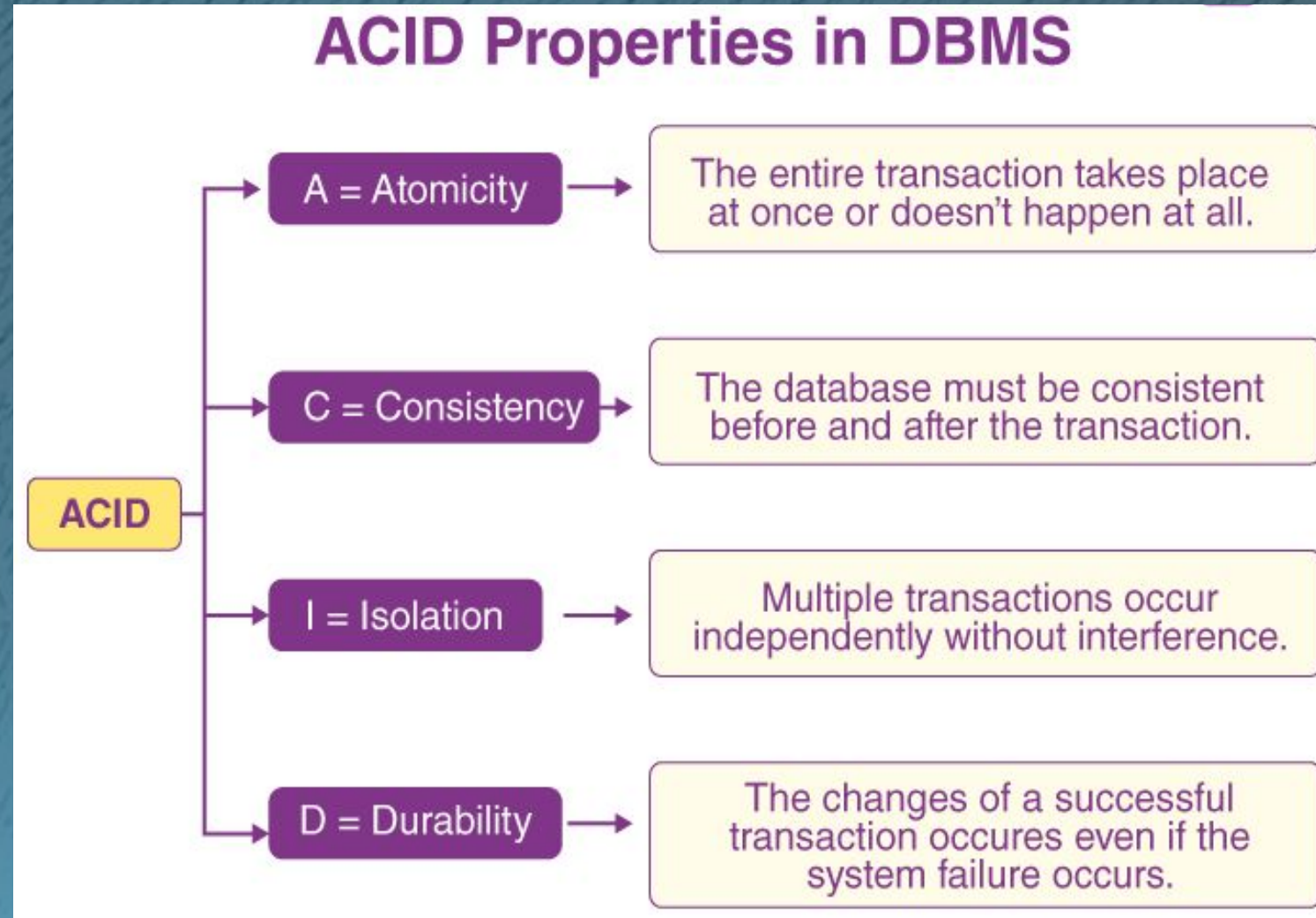
# ACID Properties in DBMS

- in DBMS that are necessary for maintaining data consistency, integrity, and reliability while performing transactions in the database.
- A **transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database. Transactions access data using read-and-write operations. To maintain consistency in a database, before and after the transaction, certain properties are followed. These are called **ACID** properties.



# A.C.I.D. properties: Atomicity, Consistency, Isolation, and Durability

- If a database operation has these ACID properties, it can be called an ACID transaction, and data storage systems that apply these operations are called **transactional systems**.
- ACID transactions guarantee that each read, write, or modification of a table has the following properties:



# A – Atomicity

- Atomicity means that an entire transaction either takes place all at once or it doesn't occur at all. It means that there's no midway. The transactions can never occur partially. Every transaction can be considered as a single unit, and they either run to completion or do not get executed at all. We have the following two operations here:
- —Commit: In case a transaction commits, the changes made are visible to us. Thus, atomicity is also called the 'All or nothing rule'.
- —Abort: In case a transaction aborts, the changes made to the database are not visible to us.
- Consider this transaction T that consists of T<sub>1</sub> and T<sub>2</sub>: Transferring 100 from account A to account B.

|                                       |                                       |
|---------------------------------------|---------------------------------------|
| Before: A : 500                       | B : 200                               |
| Transaction T                         |                                       |
| T1                                    | T2                                    |
| Read (A)<br>A: = A - 100<br>Write (A) | Read (B)<br>B: = B + 100<br>Write (B) |
| After: A : 400                        | B : 300                               |

In case the transaction fails when the T<sub>1</sub> is completed but the T<sub>2</sub> is not completed (say, after write(A) but before write(B)), then the amount has been deducted from A but not added to B. This would result in a database state that is inconsistent. Thus, the transaction has to be executed in its entirety in order to ensure the correctness of the database state.

# C – Consistency

- Consistency means that we have to maintain the integrity constraints so that any given database stays consistent both before and after a transaction. If we refer to the example discussed above, then we have to maintain the total amount, both before and after the transaction.
- Total after T occurs =  $400 + 300 = 700$ .
- Total before T occurs =  $500 + 200 = 700$ .
- Thus, the given database is consistent. Here, an inconsistency would occur when T<sub>1</sub> completes, but then the T<sub>2</sub> fails. As a result, the T would remain incomplete.

# Isolation:

- This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of the database state. Transactions occur independently without interference.
- Changes occurring in a particular transaction will not be visible to any other transaction until that particular change in that transaction is written to memory or has been committed.
- This property ensures that the execution of transactions concurrently will result in a state that is equivalent to a state achieved these were executed serially in some order.

Let  $X = 500$ ,  $Y = 500$ .

Consider two transactions  $T$  and  $T''$ .

# Cont...

Suppose that T has been executed here till Read(B) and then T'' starts. As a result, the interleaving of operations would take place. And due to this, T'' reads the correct value of A but incorrect value of B.

T'':  $(X+B = 50,000+500=50,500)$

Thus, the sum computed here is not consistent with the sum that is obtained at the end of the transaction:

T:  $(A+B = 50,000 + 450 = 50,450)$ .

It results in the inconsistency of a database due to the loss of a total of 50 units. The transactions must, thus, take place in isolation. Also, the changes must only be visible after we have made them on the main memory.

| T              | T''          |
|----------------|--------------|
| Read (X)       | Read (X)     |
| $X := X * 100$ | Read (Y)     |
| Write (X)      | $Z := X + Y$ |
| Read (Y)       | Write (Z)    |
| $Y := Y - 50$  |              |
| Write (Y)      |              |

# Durability:

- This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs.
- These updates now become permanent and are stored in non-volatile memory. The effects of the transaction, thus, are never lost.

# Some important points:

- Property Responsibility for maintaining properties
- Atomicity Transaction Manager
- Consistency Application programmer
- Isolation Concurrency Control Manager
- Durability Recovery Manager
- The **ACID** properties, provide a mechanism to ensure the correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations, and updates that it makes are durably stored.

# Advantages of ACID Properties in DBMS

- **Data Consistency:** ACID properties ensure that the data remains consistent and accurate after any transaction execution.
- **Data Integrity:** ACID properties maintain the integrity of the data by ensuring that any changes to the database are permanent and cannot be lost.
- **Concurrency Control:** ACID properties help to manage multiple transactions occurring concurrently by preventing interference between them.
- **Recovery:** ACID properties ensure that in case of any failure or crash, the system can recover the data up to the point of failure or crash.